SIMATS ENGINEERING

Department of Information Technology

CSA37 – Software Testing

ASSIGNMENT 1

CO1 – SOFTWARE QUALITY & TESTING PROCESS

EVALUATING THE QUALITY PROCESS AND DEFECT PROFILE OF MEDIQUEUE

A case-based quality engineering analysis of a hospital outpatient appointment application

Student Name	**S Deepak**	**Register Number**	**1921260077**
Department / Section	B.Tech IT	**Academic Year / Semester**	2026-2030 SEM-2
Faculty Name	R.Kesavan	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	02-11-2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	5	**5**	All four engineering features (planning, measurement, standards, repeatability) judged against case evidence and mapped to incidents; engineered approach clearly stated.
2	Evaluate the Role of Process in Software Quality	2	**2**	Clear process-over-people position; Table 2 tests each incident; 38/100 coding-defect argument and TMM Level 1 placement well justified.
3	Evaluate Testing as a Process	6	**6**	Full test process, V&V per phase, detection phase per incident; cost ₹26,10,500 computed correctly; defect-flow model and limits explicitly stated.
4	Apply and Critique Basic Definitions	4	**4**	Definitions applied to MediQueue; error/fault/failure for I3, I5, I6 correct; I8 linked to a wrong oracle; glossary contents justified.

Student Signature	Faculty Signature	Program Director

5	Evaluate the Application of Testing Principles	3	**3**	Six principles each tied to an incident or decision; “testing shows presence, not absence” used well; strongest principle defended with cost data.
6	Evaluate the Tester’s Role	3	**3**	Clear position; tester role per phase, pros/cons of self testing, 7/2/1 catch table and healthcare-specific tester qualities.
7	Trace the Origins of Defects	4	**4**	Origin and evidence for all ten incidents with second-origin reasoning; pattern (50% requirements/design) matches the defect data.
8	Evaluate Defect Classes and Severity	4	**4**	Classification scheme with sub-classes; class, severity and priority for all ten; I7 vs I1 contrast explains severity ≠ priority.
9	Design and Appraise a Defect Repository	3	**3**	Field groups justified individually; analysis/improvement/prevention uses and e-mail vs repository comparison are convincing.
10	Evaluate Defect Examples and Prevention Strategies	4	**3.5**	Three classes, root causes and prevention strategies correct. Cost model rests on assumed ₹32,000 escape cost; overlapping savings noted but not quantified.
11	Recommend and Defend a Quality Improvement Plan	4	**3.5**	Prioritised, practical plan with 3.2% effort check. Expected effect depends on assumed 60%/35%/60% efficiencies that are not validated.
12A	Re-evaluate: Impact Analysis	2	**2**	Change table and affected assumptions clear; effect of two week releases and shared records on severity well analysed.
12B	Re-evaluate: Evaluation of the Original Plan	2	**2**	Capacity check (184 vs 140 hours, 31% over) supports the conclusion; new external-system defect origins and repository gaps identified.
12C	Re-evaluate: Adaptation or Redesign	2	**2**	Clear MODIFY decision; revised two-week cycle, tester role, principles and external-defect classification (Table 26) shown.
12D	Re-evaluate: Evidence Based Defence	2	**2**	Decision defended with defect, cost, capacity and patient safety evidence; accepted risks and review trigger stated.
	TOTAL	**50**	**49**	**Percentage: 98%**

OVERALL REMARKS

Excellent, well-evidenced submission. Every task is answered with case data, the cost and defect flow calculations are correct and internally consistent (₹26,10,500; 60 production escapes; 9.6% early detection), and the Day-4 redesign follows logically from a capacity calculation. Minor weaknesses: the savings in Tasks 10–11 depend on assumed review/test efficiencies and cost figures that are stated but not validated, and the solution repeats each task's bullet list, which adds length without adding marks

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A well-argued assessment of MediQueue's testing practice against the features of an engineering activity, backed by evidence from the case.	✓	Tables 1–2 and incident mapping
2	A well-justified position on how process affects quality, backed by data from the case.	✓	TMM placement justified
3	A process-oriented assessment of testing in MediQueue, backed by cost calculations from the defect data and a reasoned estimate of savings.	✓	Calculations shown; Figure 1 present
4	Accurate application of basic testing definitions to the MediQueue incidents and a justified judgement on how terminology affects quality.	✓	I3, I5, I6 and I8 covered
5	A justified assessment of the developers' assertion using testing principles tied to evidence from the case.	✓	Six principles linked to evidence
6	A defended judgement on how valuable an independent tester would be for MediQueue, tied to specific incidents.	✓	Position defended with catch table
7	A justified origin for every incident and an interpretation of the overall pattern of defect origins.	✓	All ten incidents traced
8	A justified classification of all ten incidents by class, severity and priority, together with a recommendation on which class to address first.	✓	Severity vs priority contrast given
9	A defect repository design for MediQueue with a justification for every field and an assessment of its benefits.	✓	Each field group justified
10	A root-cause analysis of three defects with justified prevention strategies that take cost into account.	**P**	Savings rest on assumed costs
11	A technically justified quality improvement plan backed by the evidence built up in earlier tasks.	**P**	Efficiencies assumed, not validated

12A–D	Impact analysis, evaluation of the original solution, adaptation or redesign, and evidence-based defence	✓	All four parts complete

DEFECT ANALYSIS EVIDENCE CHAIN (Tasks 3, 7–10)

Evidence item	Status	Observation
Defect data calculations (cost per phase, totals)	✓	Cost per phase and totals correct
Phase-containment / detection figures	✓	Table 7 flow; 40% detection before release
Origin assigned to every incident (I1– I10)	✓	I1–I10 with evidence and second origin
Class, severity and priority justified	✓	Table 17 with reasons
Defect repository fields with reasons	✓	Table 18 with reasons
Root cause → prevention strategy link	✓	Table 20
Cost–benefit of prevention strategies	P	Ratios 12.6 and 8.8 depend on assumptions
Interpretation of the defect pattern	✓	Pareto and investment conclusion
Figures and conclusions consistent	✓	Figures consistent across Tasks 1–12

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 12 tasks (including 12A–12D) answered	☒ Yes ☐ Partly ☐ No
Defect cost calculations shown step by step	☒ Yes ☐ Partly ☐ No
All ten incidents classified (origin, class, severity, priority)	☒ Yes ☐ Partly ☐ No
Error, fault and failure separated for I3, I5 and I6	☒ Yes ☐ Partly ☐ No
Testing principles linked to evidence from the case	☒ Yes ☐ Partly ☐ No
Repository design justified field by field	☒ Yes ☐ Partly ☐ No
Tables, figures and diagrams included and readable	☒ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
No unfilled template placeholders	☒ Yes ☐ Partly ☐ No

Viva recommended	☐ Yes ☒ No

SUBMISSION OVERVIEW

This document answers all twelve tasks in the order and mark distribution of the supplied question paper. The analysis uses the stated incidents and defect data. Estimates are labelled clearly where the brief does not provide defect-by-defect detection histories.

CASE STUDY SNAPSHOT

CASE FACT	DETAIL
Application	MediQueue – mobile and web outpatient appointment application
Usage	Approximately 6,000 patients per day
Development	Eight months; five developers; no separate tester
Original testing	Developers tested their own code just before release; approval followed a management demo
Reported outcome	Ten incidents reported within three months after release
Future change	Two-week releases and connection to the national digital health ID system

Overall finding: the problem is systemic. Defects originated in requirements, design, coding, testing and documentation. The absence of formal reviews and repeatable, independent testing allowed many defects to reach patients.

CONTENTS

TASK	TOPIC	MARKS
1	Testing as an engineering activity	5
2	Role of process in software quality	2
3	Testing as a life-cycle process and cost analysis	6
4	Basic testing definitions	4
5	Software testing principles	3
6	Tester's role	3
7	Origins of defects	4
8	Defect classes, severity and priority	4
9	Defect repository	3
10	Defect examples and prevention	4
11	Quality improvement plan	4
12	Day-4 design challenge	8

TASK 1 | 5 MARKS

Evaluate Testing as an Engineering Activity

CO1 | Bloom's Level: K5 – Evaluate

MediQueue's testing cannot reasonably be called a complete engineering activity. The team performed some checking, but it did not show the planned, measured, standards-based and repeatable discipline expected of engineering. A successful management demonstration is not objective evidence that the product meets all requirements or quality targets.

Engineering characteristics compared with the case

CHARACTERISTIC	ENGINEERING EXPECTATION	MEDIQUEUE EVIDENCE AND EFFECT
Planning	Define scope, risks, responsibilities, schedule, environments and entry/exit criteria before testing.	No separate tester or formal test plan is described. Testing occurred just before release, so concurrency (I1, I6), boundary (I4), business-rule (I2, I5) and integration (I9) risks were not planned systematically.
Measurement	Use coverage, defect trends, performance results, severity and acceptance criteria to judge quality.	A demo with no visible errors was treated as sufficient. There is no evidence of requirement coverage or performance thresholds; the 25-second load (I10) escaped.
Standards	Use common procedures, review checklists, coding rules and approved expected results.	No formal requirements/design reviews are stated. I7 shows documentation was not reconciled with the interface; I8 shows the expected refund value was not validated.
Repeatability	Document and version tests, control data/environments and repeat regression tests.	Ad-hoc developer checks and a demo are not a repeatable regression suite. I1/I6 need concurrency tests; I9 needs repeatable API-key rotation tests.

How an engineered approach would look

• Before development, agree a risk-based test strategy with hospital stakeholders and define measurable acceptance criteria for appointments, fees, priority, refunds and performance.

• Review requirements for missing rules and testability; inspect designs for concurrency, data isolation, integration and performance risks.

• During coding, use peer review, static checks, unit tests and boundary-value tests.

• Run integration and system tests for payment, SMS, token allocation and prescription generation, including simultaneous transactions and failure paths.

• Maintain a controlled regression suite and approve release only against traceable evidence and agreed exit criteria.

Conclusion: the team hoped quality would emerge from developer checks and a successful demo. Engineering quality must be deliberately planned, measured and verified throughout the life cycle.

TASK 2 | 2 MARKS

Evaluate the Role of Process in Software Quality

CO1 | Bloom's Level: K5 – Evaluate

Position: quality depends on capable people working within a defined process. Skilled developers are important, but skill alone cannot replace requirement clarification, reviews, independent challenge, repeatable testing or defect learning.

A defined process assigns responsibilities, establishes checkpoints, ensures requirements are testable, links tests to requirements, records defects consistently and prevents release when critical risks remain unresolved. It also creates feedback: recurring defects can be analysed and the process improved.

Better developers alone would not necessarily prevent these incidents. A programmer cannot reliably enforce a Sunday closure rule that was never supplied (I2). GST interpretation (I3) requires agreement with the hospital's finance owner. I7 is a mismatch between the manual and interface; I8 is a test-oracle problem. I1 and I6 require deliberate concurrency design and testing.

Process maturity judgement

Using the Testing Maturity Model (TMM) as a qualitative guide, the team is closest to **Level 1 – Initial**: testing is ad hoc, reactive and dependent on individual developers. There is no evidence of a managed test strategy, formal reviews, defined test organisation or systematic measurement. This is a case-based judgement, not a formal TMM audit.

Recommendation: retain the five developers but introduce a lightweight process with named quality ownership, peer review, test planning, traceability, defect tracking and release criteria. A process makes good people's work consistent.

TASK 3 | 6 MARKS

Evaluate Testing as a Process

CO1 | Bloom's Level: K5 – Evaluate

Testing should run through the whole life cycle rather than being a single event immediately before release. Its activities include planning, test analysis/design, implementation and environment preparation, execution, evaluation and reporting.

Verification and validation across development

PHASE	ACTIVITY	MEDIQUEUE DEFECTS TO CATCH
Requirements	Review completeness, consistency, testability and stakeholder agreement; validate workflows.	I2 Sunday closure; I3 GST/fee rule; I5 age threshold and effective date; I7 cancellation workflow.
Design	Inspect architecture, data flow, concurrency, privacy boundaries and interface contracts.	I1 token uniqueness; I6 patient-context isolation; I9 API-key handling; I10 performance architecture.
Coding	Peer review, static analysis, unit tests and boundary checks.	I4 long-name handling; I5 date calculation; I3 tax calculation; I6 shared-state handling.
Integration / system	Test workflows, interfaces, concurrency, negative paths, performance and regression.	I1 simultaneous booking; I6 concurrent PDF generation; I9 provider key rotation; I10 hospital Wi-Fi performance.
Acceptance / documentation	Validate user workflows and consistency between manual and released interface.	I7 manual/menu mismatch; confirm fee, priority, refund and appointment rules.

Cost of late detection – calculation

The supplied table gives the number of defects detected in each phase and the cost to fix one defect at that phase. Estimated cost = defects detected × cost per defect.

DETECTION PHASE	DEFECTS	COST / DEFECT	ESTIMATED COST
Requirements	2	■ 500	■ 1,000
Design	3	■ 1,500	■ 4,500
Coding	15	■ 3,000	■ 45,000
Testing	20	■ 8,000	■ 1,60,000
Production	60	■ 40,000	■ 24,00,000
TOTAL	100	—	■ 26,10,500

Interpretation: 60% of defects were detected only after release. Production correction accounts for ■24,00,000, about 91.9% of the estimated total ■26,10,500. This excludes support effort, patient dissatisfaction, reputational damage and possible privacy/clinical consequences.

Illustrative saving from earlier reviews

Requirements introduced 30 defects, but only 2 were detected in that phase; therefore 28 escaped that phase. Design introduced 22, but only 3 were detected there; therefore 19 escaped. If reviews had caught these 47 defects at origin, indicative correction cost would be (28 × ■500) + (19 × ■1,500) = ■14,000 + ■28,500 = ■**42,500**.

For an upper-bound comparison only, if all 47 had otherwise incurred the stated production rate, cost would be 47 × ■40,000 = ■18,80,000. The illustrative difference is ■18,80,000 – ■42,500 = ■**18,37,500**. This is not a guaranteed saving: the table does not identify the eventual detection phase of each individual defect, and not every escaped defect necessarily reached production. The calculation assumes phase costs apply uniformly.

Conclusion: the weighted estimate is ■26.105 lakh, with most cost concentrated after release. Requirements and design reviews are likely to provide high leverage, but the exact saving cannot be known from the supplied aggregate data.

TASK 4 | 4 MARKS

Apply and Critique Basic Definitions

CO1 | Bloom's Level: K5 – Evaluate

Testing terms are operational, not interchangeable labels. Distinguishing a human mistake, the defect left in an artefact and the observable failure helps the team assign the correct corrective action.

TERM	MEANING IN MEDIQUEUE
Error	A human action or misunderstanding that produces an incorrect result. Example: misunderstanding how GST should be applied (I3).
Fault / defect	The incorrect requirement, design, code or document left in the product. Example: incorrect fee/tax calculation logic (I3).
Failure	The observable behaviour when the system runs. Example: patient sees ■500 but is charged ■550 (I3).
Test case	Documented preconditions, inputs, steps and expected results. Example: two simultaneous bookings should receive unique tokens.
Test oracle	The source used to decide whether output is correct. For fees/refunds, this must be an approved hospital/finance rule.
Test bed	The controlled hardware, software, network, data and configuration used to execute tests, including payment/SMS sandbox and hospital-like Wi-Fi.

Error–fault–failure for I3, I5 and I6

INCIDENT	ERROR	FAULT	FAILURE
I3	GST rule misunderstood.	Incorrect or inconsistent fee/tax logic.	Page shows ■500 while ■550 is charged.
I5	Age eligibility rule interpreted using the wrong date/boundary.	Eligibility logic fails to evaluate age on appointment day.	Eligible patient does not receive senior priority.
I6	Design/coding failed to isolate each doctor's patient context.	Shared or stale patient data/state reused during PDF generation.	PDF shows the previous patient's details.

Why I8 happened; required glossary

In I8, the refund test was marked Pass against an incorrect expected value. The test may have executed as written, but its oracle was wrong. Calling every issue a "bug" can hide whether the product, test case or expected result is defective. A common glossary should define error, fault/defect, failure, incident, severity, priority, test case, test data, oracle, test bed, verification, validation, retest, regression and closure criteria, each with a MediQueue example.

Judgement: terminology affects quality because it determines root-cause assignment, ownership and prevention. I8 shows that an incorrect oracle can make a defective system appear correct.

TASK 5 | 3 MARKS

Evaluate the Application of Testing Principles

CO1 | Bloom's Level: K5 – Evaluate

The developers' claim that "we tested everything" is not defensible. Testing can reveal defects, but cannot prove that no defects remain.

PRINCIPLE	APPLICATION TO MEDIQUEUE
Testing shows presence of defects, not their absence.	I1–I10 appeared after release despite the pre-release demo. A clean demo proves only that demonstrated scenarios worked.
Exhaustive testing is impossible.	There are many dates, names, patients, doctors, payment states, simultaneous bookings and provider configurations. Risk-based testing is needed.
Early testing saves time and cost.	No formal requirements/design reviews were conducted; 60 defects were found in production at ■ 40,000 each.
Defects cluster together.	The incidents cluster around business rules, concurrency, integration and test-oracle quality; these areas need focused attention.
Pesticide paradox.	Repeating a narrow management demo will not expose long names, concurrent saves or API-key rotation. Test scenarios must evolve.
Testing is context-dependent.	A healthcare appointment and prescription system needs strong correctness, privacy, concurrency and availability checks.

Most influential principle: early testing saves time and cost. Requirements and design introduced 52 defects, but only five were detected in those phases. Reviews could have exposed I2 and I5 before coding and clarified I3 before payment logic was implemented. This principle does not replace integration or concurrency testing; it shifts quality work earlier.

TASK 6 | 3 MARKS

Evaluate the Tester's Role

CO1 | Bloom's Level: K5 – Evaluate

Position: an independent tester would probably have reduced escaped defects, but would not automatically have prevented every incident. Independence adds a different perspective and challenges assumptions; it must be supported by early involvement and access to requirements.

Tester responsibilities through the life cycle

• Review requirements for ambiguity, missing rules and testability; clarify Sunday operations, GST, senior eligibility and cancellation steps.

• Review designs for concurrency, patient-data separation, security, integration and performance.

• Design and execute unit, integration, system, acceptance, boundary, negative, concurrency, performance and regression tests.

• Maintain controlled test data and test beds; verify test oracles against approved business rules.

• Record defects with evidence, severity and reproducible steps; retest fixes and run regression tests.

• Report coverage, unresolved risks and release readiness independently of delivery pressure.

Developer testing: advantages and limitations

Developers understand implementation details and can write fast unit tests close to the code. This is efficient for logic and boundary checks. However, developers may share the same assumptions as the author, focus on intended behaviour rather than user workflows, or lack time to challenge requirements. I8 shows that test execution is not enough when the expected result is wrong.

Likely catches

INCIDENTS	LIKELY TEST / STAGE
I2, I3, I5	Requirements review and acceptance-test design with hospital/finance stakeholders.
I1, I6	Design review plus integration/system concurrency tests with simultaneous transactions and separate patient records.
I4	Unit/system boundary tests using names below, at and above 40 characters.
I7	Acceptance testing and documentation review against the actual interface.
I8	Test-case peer review and validation of expected results against an approved refund rule.
I9	Integration/operational tests for API-key rotation, authentication failure and recovery.
I10	Performance testing against a defined response-time target on representative hospital Wi-Fi.

A healthcare tester should be analytical, sceptical, detail-oriented, ethical and able to communicate with clinicians, administrators and developers. They must protect confidentiality, use safe test data, understand patient risk, document evidence and escalate critical defects.

TASK 7 | 4 MARKS

Trace the Origins of Defects

CO1 | Bloom's Level: K5 – Evaluate

The origin is the earliest life-cycle artefact or condition that created the defect, not necessarily the phase in which it was noticed. Some incidents have multiple contributing origins; the primary origin below is selected from the strongest case evidence.

ID	PRIMARY ORIGIN	EVIDENCE / CONTRIBUTING FACTOR
I1	Design / coding	Simultaneous bookings create duplicate tokens; likely missing atomic allocation, uniqueness constraint or concurrency control.
I2	Requirements	The brief explicitly says the Sunday closure rule was never written.
I3	Requirements / design	GST was misunderstood; unclear business rule may have led to inconsistent fee display and charge logic.
I4	Coding	Long names cause a crash, suggesting inadequate input handling, storage limits or boundary checks.
I5	Requirements / coding	The age-on-appointment-day boundary was not correctly specified or translated into date-based logic.
I6	Design / coding	Concurrent saves expose stale/shared patient context during prescription PDF generation.
I7	Documentation	The manual refers to a menu absent from the product; documentation was not checked against the released interface.
I8	Testing	The refund test's expected value was wrong and not independently validated.
I9	Integration / environment	Provider API-key change stops SMS; configuration and external-interface resilience were not handled/tested.
I10	Design / performance	25-second load suggests inefficient query/data handling or unsuitable performance architecture; performance acceptance was absent.

Overall pattern and investment

The phase data show 30 defects originated in requirements, 22 in design, 38 in coding, five in testing and five in documentation. Requirements plus design account for 52% of defects. The detection problem is also severe: 60 of 100 were found in production. The hospital should invest first in requirements clarification and design review, while strengthening integration, concurrency and performance testing. Coding standards matter, but cannot correct missing business rules or an invalid oracle.

TASK 8 | 4 MARKS

Evaluate Defect Classes and Severity

CO1 | Bloom's Level: K5 – Evaluate

Severity estimates the consequence if a defect occurs; priority determines how urgently it should be fixed, considering exposure, frequency, workarounds and release context. Ratings below are reasoned recommendations because the brief supplies no formal severity scale or frequency data.

ID	CLASS / SUBCLASS	SEVERITY	PRIORITY	JUSTIFICATION
I1	Design/coding – concurrency	Critical	P1	Duplicate tokens can disrupt patient queues and appointments, especially under simultaneous bookings.
I2	Requirements – missing rule	High	P1	Invalid Sunday bookings cause failed visits and operational disruption.
I3	Requirements/design – financial rule	High	P1	Patient sees one fee and is charged another, damaging trust and creating financial disputes.
I4	Coding – boundary/robustness	Medium	P2	Long-name input crashes the app for affected users; frequency is unknown.
I5	Requirements/coding – date boundary	High	P1	Eligible senior patients lose priority; fairness and access concerns.
I6	Design/coding – data isolation/privacy	Critical	P1	Prescription may expose another patient's details; serious confidentiality and clinical risk.
I7	Documentation – UI mismatch	Low–Medium	P2	Cancellation instructions fail; workaround may exist, but missed cancellations are possible.
I8	Testing – incorrect oracle	High	P1	False pass can allow refund defects to escape; actual financial impact is unknown.
I9	Integration/configuration	High	P1	Reminders fail after key rotation; may cause missed appointments.
I10	Performance	High	P1	25-second schedule load impairs clinicians' workflow and may worsen under peak load.

Why severity and priority may differ

I1 is critical in severity because it compromises token uniqueness and can affect several patients; it is also P1 because it should be fixed urgently. I7 may be lower in severity because a workaround might exist, but its priority could rise if cancellations are time-sensitive or the workaround is unknown. Severity describes impact; priority describes urgency.

First class to target: requirements/specification defects. Thirty originated in requirements, but only two were detected there; I2, I3 and I5 show the consequence of unclear business rules. In parallel, I1 and I6 need immediate containment because their patient impact is critical.

TASK 9 | 3 MARKS

Design and Appraise a Defect Repository

CO1 | Bloom's Level: K5 – Evaluate

Email and spreadsheets fragment information, use inconsistent fields, obscure ownership and status, make duplicates difficult to identify and limit trend analysis. A repository should provide one traceable record from discovery through closure and learning.

REPOSITORY FIELD	WHY IT IS NEEDED
Unique ID and title	Unambiguous reference for discussion, duplicate detection and release notes.
Date, reporter and source	Shows when and where found: patient complaint, tester, support or monitoring.
Module, build and environment	Identifies affected booking, payment, token, prescription, SMS or schedule component and supports reproduction.
Steps, actual and expected result	Allows developers/testers to reproduce the failure and understand the mismatch.
Evidence and test-data reference	Stores screenshots, logs and correlation IDs; use masked/synthetic data rather than unnecessary patient identifiers.
Origin phase and defect class	Enables analysis of requirements, design, coding, testing, documentation and integration causes.
Severity and priority	Separates impact from urgency and supports risk-based triage.
Owner, status and timestamps	Shows responsibility, ageing, hand-offs and resolution history.
Root cause and corrective action	Records why it happened and what change fixes it.
Preventive action / lesson	Ensures the team updates a checklist, requirement, design rule, test or training.
Fix version, retest and regression	Confirms the fix is in a named build and related behaviour still works.
Related requirement, test and release	Maintains traceability and exposes coverage gaps or repeated escapes.

Repository workflow and benefits

Report → validate/duplicate-check → classify/prioritise → assign → fix → peer review → retest → regression → close → analyse trends. Review defect origin, class, severity, escape phase, reopen rate and recurring root causes after each release. Apply access controls and data minimisation: do not copy real patient identifiers or prescription contents into ordinary defect records.

Conclusion: the repository turns complaints into engineering information. It supports prevention only when root causes are analysed and preventive actions are implemented.

TASK 10 | 4 MARKS

Evaluate Defect Examples and Prevention Strategies

CO1 | Bloom's Level: K5 – Evaluate

Three incidents from different defect classes are selected. The effort comparison is qualitative because the brief does not provide staff rates or intervention hours.

INCIDENT	ROOT CAUSE	ONE PROCESS CHANGE	EFFORT VS BENEFIT
I2 – Sunday appointments	The closure rule was omitted from requirements.	Mandatory business-rule walkthrough and signed requirements checklist with hospital operations before design.	Low effort: short stakeholder review. High benefit: prevents invalid bookings and rework; requirements correction is priced at ■500/defect.
I1 – Duplicate tokens	Token allocation is not safe under simultaneous bookings.	Require design inspection of shared-resource operations, with documented concurrency control and a database uniqueness rule.	Moderate effort: design review and focused test. High benefit: prevents duplicate queue positions; production correction is ■40,000/defect.
I8 – Refund false pass	Expected refund value was wrong and not independently checked.	Require peer review of test cases against a finance-approved oracle before execution.	Low effort: second-person review. High benefit: prevents false assurance and incorrect refund behaviour escaping.

Best value for the hospital

The requirements walkthrough likely gives the best overall value: it is inexpensive and addresses a major upstream source (30 requirements-origin defects, only two detected in that phase). However, the concurrency design inspection is an immediate operational priority because I1 can affect multiple patients and production fixes cost ■40,000 each. Oracle review is also inexpensive and directly addresses I8.

Recommendation: make requirements walkthroughs standard for every feature, while immediately adding concurrency design checks and oracle review to the quality checklist. Best value considers prevention benefit and risk, not only the cheapest action.

TASK 11 | 4 MARKS

Recommend and Defend a Quality Improvement Plan

CO1 | Bloom's Level: K5 – Evaluate

Adopt a lightweight, risk-based process suitable for five developers. Quality ownership can rotate, but testing and release approval must include review by someone other than the author.

PRIORITY	PROCESS CHANGE	OWNER / EVIDENCE
1 – Immediate	Contain I1 and I6; fix and regression-test token uniqueness and patient-data isolation.	Technical lead + independent reviewer; defect records and concurrency/privacy tests.
2 – Every feature	Clarify business rules; record acceptance criteria, edge cases and approved financial/eligibility rules.	Feature owner + hospital representative; signed requirements and test links.
3 – Before completion	Peer-review designs for concurrency, integration, data handling and performance; review code and unit tests.	Rotating reviewer; design/code checklist and unit-test results.
4 – Every integration	Repeatable integration tests for payment, SMS, token and PDF; include failure and credential-change paths.	Integration owner; logs, API contract version and regression results.
5 – Each release	Risk-based system, acceptance, boundary, performance and regression tests; report unresolved risks.	Rotating test lead; traceability, defect summary and release sign-off.
6 – Continuous	Maintain repository and analyse escaped defects; update checklists based on root causes.	Quality coordinator; trend report and preventive actions.

Expected effect, trade-offs and feasibility

The plan should reduce omissions through early review, catch implementation faults through unit/integration testing and prevent false assurance through oracle review. The strongest numerical signal is that production detection accounts for 60 defects and ■24,00,000 of the estimated ■26,10,500 correction cost. Moving detection earlier should reduce production escapes and correction costs. An exact future defect count cannot be promised; measure escaped-defect rate, severity-weighted defects, detection phase, reopen rate, coverage and time to fix.

Reviews and tests consume time and may initially slow delivery. A five-person team may struggle with a large suite or full-time QA role. Use rotating ownership, short checklists, automation for stable high-value regression tests and part-time independent review for high-risk changes. Automation itself needs maintenance; brittle tests can create false alarms.

Conclusion: use a staged, lightweight quality process rather than a heavyweight framework. Five developers can operate it if roles rotate, risk is prioritised and release decisions use evidence instead of a demo.

TASK 12 | 8 MARKS

Re-evaluate and Adapt the Quality Process – Day-4 Design Challenge

CO1 | Bloom's Level: K5 – Evaluate

Final decision: MODIFY the original plan. Its core controls remain necessary, but two-week releases and cross-hospital health-record exchange introduce new delivery, integration, privacy and operational risks. A full redesign is not required if the process becomes continuous and stronger at external-system boundaries.

A. Impact analysis – 2 marks

• **Release frequency:** a two-week cadence leaves less time for a large end-stage test phase. Reviews, unit tests and regression checks must happen throughout the sprint.

• **External dependency:** the national digital health ID system is maintained by another agency. API versions, authentication, availability, data formats, rate limits, certificates and change notices may be outside the hospital team's control.

• **Health-record sharing:** incorrect identity matching or patient-data association can expose information across hospitals. I6-like data-isolation failures become more serious; privacy, access control, consent and auditability need explicit testing.

• **Changed assumptions:** manual regression of every workflow may not fit each cycle; prioritised automated regression and early integration testing are needed.

B. Evaluation of the original plan – 2 marks

Reviews and tests can fit a two-week cycle if they are continuous rather than large end-stage gates. Refine requirements before/during sprint planning; review designs and code as work completes; run automated unit and integration tests with each change; execute a short risk-based system/regression suite before release; and review acceptance evidence before deployment.

New defect origins include external API contract changes, identity matching errors, authentication/token expiry, certificate rotation, network outages, rate limiting, schema incompatibility, data mapping errors, duplicate/delayed messages, consent/authorisation mismatch and incorrect patient-record linkage. Classify these as integration, interface, configuration, security/privacy, data-quality or external-dependency defects, while recording internal contributions.

The repository remains a good foundation but needs additional fields: external service/agency, API/schema version, request/correlation ID, timestamp/time zone, response code, retry behaviour, data-mapping version, consent/authorisation context, affected record type, environment and internal/external/joint cause. Never store secrets such as API keys.

C. Adaptation / redesign – 2 marks

CYCLE POINT	REVISED QUALITY ACTIVITY
Backlog refinement	Confirm acceptance criteria, data-sharing purpose, consent and privacy requirements; identify external dependencies.
Design before implementation	Review API contracts, identity matching, data mapping, security boundaries, concurrency and retry/failure design.
During development	Peer review, unit tests, static checks and automated contract tests with each change; use synthetic/masked data.
Continuous integration	Run API contract, integration and regression tests; simulate timeouts, key/certificate rotation and service unavailability.
Pre-release	Run risk-based end-to-end, privacy/security, performance and recovery tests; review critical defects and rollback readiness.
Post-release	Monitor failures, latency, authentication errors and data-integrity signals; feed incidents into the next sprint.

The tester becomes embedded and continuous: participate in backlog refinement, define acceptance tests early, maintain automated regression/contract tests, review test-data privacy, test failure modes and monitor production quality signals. Independence is preserved through peer review and a release-risk review by someone other than the feature author.

Testing principles gaining importance

Risk-based testing is more important because health-record privacy and identity linkage have high consequences. Early testing is essential because a two-week release cannot absorb late discovery. Testing is context-dependent: security, privacy, interoperability and data integrity need greater emphasis. Defect clustering and the pesticide paradox also matter: recurring integration failures require new scenarios, not only rerunning happy paths.

D. Evidence-based defence – 2 marks

The decision is supported by the original data: 30 requirements defects and 22 design defects, with only five detected in those phases, justify retaining early reviews. Sixty defects were detected after release at ■40,000 each, making continuous integration and regression economically important. I1 and I6 show the need for concurrency and data-isolation testing. External health ID integration increases the consequence of incorrect identity or record association.

A complete replacement process would add unnecessary disruption for five developers. Retaining the original plan unchanged would be inadequate for two-week delivery and external health-data exchange. Modification preserves the useful controls while adding automated checks, contract testing, privacy controls, monitoring and dependency tracking.

Final conclusion: MODIFY. Keep requirements reviews, design reviews, independent testing and the defect repository, but make them continuous and sprint-based. Add integration-contract, identity, consent, privacy, data-integrity, resilience and monitoring controls before sharing records across hospitals.

FINAL QUALITY ENGINEERING SUMMARY

MediQueue's failures are best explained by a weak quality process rather than one isolated coding problem. The data show 52 defects originated in requirements and design, only five were detected in those phases, and 60 were discovered after release. The estimated correction cost based on detection phase is ■26,10,500, of which ■24,00,000 is attributed to production detection. These figures justify moving quality work earlier and making testing continuous.

KEY FINDING	ACTION
Requirements and design are major defect sources.	Conduct stakeholder walkthroughs and design inspections before implementation.
Production escapes dominate cost.	Use risk-based system, integration and regression testing with evidence-based release criteria.
Concurrency and patient-data isolation are critical.	Test simultaneous transactions, atomic token allocation and strict patient-context isolation.
A false pass is possible with a wrong oracle.	Peer-review test cases and use approved business rules as expected results.
Two-week releases and health-ID integration raise complexity.	Adopt continuous integration, contract tests, privacy controls, monitoring and dependency tracking.

STUDENT SUBMISSION CHECKLIST

- ■ Cover page completed with student, register number, department, faculty and submission date.
- ■ All Tasks 1–12 answered in the order of the question paper.
- ■ Cost calculations include assumptions and limitations.
- ■ All ten incidents classified and traced to likely origins.
- ■ Tables, terminology and page layout checked before submission.
- ■ Final recommendation supported by defect data, cost, risk and team capacity.

END OF ASSIGNMENT